

Солдатов Владимир ИСИП-406 Практическая работа 2

1. Функции высшего порядка + ООП

Цель: Написать класс MathProcessor, который:

1. Принимает в конструкторе список чисел List<double>.
2. Имеет метод process, принимающий функцию-обработчик (например, возведение в квадрат) и возвращающий новый список.
3. Реализует статический метод getRandomNumbers, генерирующий список случайных чисел.

```
C: > Users > SoW > Desktop > FlutterSDK > flutter_2_1.dart > MathProcessor

1  import 'dart:math';
2
3  class MathProcessor {
4      final List<double> numbers;
5
6      MathProcessor(this.numbers);
7
8      /// Метод высшего порядка:
9      /// принимает функцию-обработчик и возвращает новый список
10     List<double> process(double Function(double) handler) {
11         return numbers.map(handler).toList();
12     }
13
14     /// Статический метод генерации случайных чисел
15     static List<double> getRandomNumbers(int count,
16         {double min = 0, double max = 100}) {
17         final random = Random();
18         return List.generate(
19             count,
20             (_) => min + random.nextDouble() * (max - min),
21         );
22     }
23 }
24
25 void main() {
26     // Генерация случайных чисел
27     List<double> data = MathProcessor.getRandomNumbers(10, min: 1, max: 50);
28 }
```

C: > Users > SoW > Desktop > FlutterSDK > flutter_2_1.dart > MathProcessor

```
23 }
24
   Run | Debug
25 void main() {
26     // Генерация случайных чисел
27     List<double> data = MathProcessor.getRandomNumbers(10, min: 1, max: 50);
28
29     // Создание объекта
30     MathProcessor processor = MathProcessor(data);
31
32     print("Original:");
33     print(data);
34
35     // Пример 1: квадрат
36     print("\nSquared:");
37     print(processor.process((x) => x * x));
38
39     // Пример 2: умножение на 2
40     print("\nDoubled:");
41     print(processor.process((x) => x * 2));
42
43     // Пример 3: округление
44     print("\nRounded:");
45     print(processor.process((x) => x.roundToDouble()));
46 }
```

PS C:\> dart run flutter_2.1.dart

Original:

[10.894325897895747, 10.15110734697824, 3.42525076273117, 18.265028765100585, 29.482280163927932, 16.76469295547349, 22.560855548845122, 40.01164639243716, 24.52131151841368, 1.0246084896983883]

Squared:

[118.68633676956198, 103.04498036987559, 11.732342787590461, 333.6112757899518, 869.2048436643385, 281.05492989130244, 508.99220309585576, 1600.9318470334294, 601.2947185830874, 1.0498225571618485]

Doubled:

[21.788651795791495, 20.30221469395648, 6.85050152546234, 36.53005753020117, 58.964560327855864, 33.52938591094698, 45.121711097690245, 80.02329278487431, 49.04262303682736, 2.0492169793966166]

Rounded:

[11.0, 10.0, 3.0, 18.0, 29.0, 17.0, 23.0, 40.0, 25.0, 1.0]

PS C:\> 

2. Абстрактные классы + Наследование

Цель: Реализовать иерархию классов для геометрических фигур:

1. Абстрактный класс Shape с методом area().
2. Классы Circle и Rectangle, наследующие Shape.
3. Метод Shape.fromJson(Map<String, dynamic> json), который создает Circle или Rectangle на основе данных JSON.

Условия:

Для Circle JSON: {"type": "circle", "radius": 5}.

Для Rectangle JSON: {"type": "rectangle", "width": 4, "height": 6}.

```
C: > Users > SoW > Desktop > FlutterSDK > flutter_2_2.dart > Rectangle >
1  import 'dart:math';
2
3  Run | Debug
4  void main() {
5      // JSON для круга
6      Map<String, dynamic> circleJson = {
7          "type": "circle",
8          "radius": 5
9      };
10
11     // JSON для прямоугольника
12     Map<String, dynamic> rectJson = {
13         "type": "rectangle",
14         "width": 4,
15         "height": 6
16     };
17
18     Shape circle = Shape.fromJson(circleJson);
19     Shape rectangle = Shape.fromJson(rectJson);
20
21     print("Circle area: ${circle.area()}");
22     print("Rectangle area: ${rectangle.area()}");
23 }
24
25 /// Абстрактный класс
26 abstract class Shape {
27     double area();
28
29     /// Фабричный метод создания объектов
30     factory Shape.fromJson(Map<String, dynamic> json) {
31         switch (json["type"]) {
32             case "circle":
33                 return Circle(json["radius"].toDouble());
34             case "rectangle":
35                 return Rectangle(
36                     json["width"].toDouble(),
37                     json["height"].toDouble());
38             default:
39                 throw ArgumentError("Unsupported type: $json["type"]");
40         }
41     }
42 }
```

C: > Users > SoW > Desktop > FlutterSDK > flutter_2_2.dart > Rectangle > area

```
25  abstract class Shape {
26
28      /// Фабричный метод создания объектов
29      factory Shape.fromJson(Map<String, dynamic> json) {
30          switch (json["type"]) {
31              case "circle":
32                  return Circle(json["radius"].toDouble());
33              case "rectangle":
34                  return Rectangle(
35                      json["width"].toDouble(),
36                      json["height"].toDouble(),
37                  );
38              default:
39                  throw Exception("Unknown shape type");
40          }
41      }
42  }
43
44  /// Круг
45  class Circle implements Shape {
46      final double radius;
47
48      Circle(this.radius);
49
50      @override
51      double area() {
52          return pi * radius * radius;
53      }
54  }
55
56  /// Прямоугольник
57  class Rectangle implements Shape {
58      final double width;
59      final double height;
60  }
```

```
C: > Users > SoW > Desktop > FlutterSDK > flutter_2_2.dart > Rectangle > area
57  class Rectangle implements Shape {
58      final double width;
59      final double height;
60
61      Rectangle(this.width, this.height);
62
63      @override
64      double area() {
65          return width * height;
66      }
67  }
```

```
63      @override
64      double area() {
65          return width * height;
66      }
67  }
```

ПРОБЛЕМЫ ВЫХОДНЫЕ ДАННЫЕ КОНСОЛЬ ОТЛАДКИ ТЕРМИНАЛ ПОРТЫ

```
PS C:\> dart run flutter_2_2.dart
Circle area: 78.53981633974483
Rectangle area: 24.0
PS C:\> 
```

3. Статические методы + Константы

Цель: Реализовать класс PasswordValidator с:

1. Константными требованиями к паролю: Минимум 8 символов. Хотя бы одна цифра.
2. Статическим методом validate, проверяющим пароль.

```
C: > Users > SoW > Desktop > FlutterSDK > flutter_2_3.dart > PasswordValidator
Run | Debug
1 void main() {
2     print(PasswordValidator.validate("1234567")); // false (мало символов)
3     print(PasswordValidator.validate("password")); // false (нет цифры)
4     print(PasswordValidator.validate("pass1234")); // true
5 }
6
7 /// Класс проверки пароля
8 class PasswordValidator {
9     /// Константы требований
10    static const int minLength = 8;
11
12    /// Статический метод проверки пароля
13    static bool validate(String password) {
14        /// 1. Проверка длины
15        if (password.length < minLength) {
16            return false;
17        }
18
19        /// 2. Проверка наличия хотя бы одной цифры
20        final hasDigit = password.contains(RegExp(r'\d'));
21
22        return hasDigit;
23    }
24 }
```

ПРОБЛЕМЫ ВЫХОДНЫЕ ДАННЫЕ КОНСОЛЬ ОТЛАДКИ ТЕРМИНАЛ ПОРТЫ

```
PS C:\> dart run flutter_2_3.dart
false
false
true
PS C:\> 
```

4. Банковская система

Условие:

- Реализуйте систему банковских счетов с использованием ООП. Требуется:
- Класс BankAccount
 - о Поля:
 - accountNumber (уникальный номер счета)
 - balance (текущий баланс, начальное значение = 0)
 - ownerName (имя владельца)
 - о Методы:
 - deposit(amount) – пополнение счета (нельзя внести отрицательную сумму).
 - withdraw(amount) – снятие денег (нельзя снять больше, чем есть на счету).
 - displayBalance() – вывод текущего баланса.
 - о Дополнительно:
 - При создании счета генерировать случайный accountNumber (например, из 10 цифр).
- Класс Bank
 - о Поля:
 - List<BankAccount> accounts (список всех счетов).
 - о Методы:
 - addAccount(ownerName) – создает новый счет и добавляет его в accounts.
 - findAccount(accountNumber) – возвращает счет по номеру (или null, если не найден).
 - transfer(fromAccountNumber, toAccountNumber, amount) – перевод денег между счетами.

C: > Users > SoW > Desktop > FlutterSDK > flutter_2_4.dart > Bank

```
1  import 'dart:math';
2
   Run | Debug
3  void main() {
4      Bank bank = Bank();
5
6      var acc1 = bank.addAccount("Alice");
7      var acc2 = bank.addAccount("Bob");
8
9      acc1.deposit(100);
10     acc1.withdraw(30);
11
12     bank.transfer(acc1.accountNumber, acc2.accountNumber, 50);
13
14     acc1.displayBalance();
15     acc2.displayBalance();
16 }
17
18 class BankAccount {
19     final String accountNumber;
20     final String ownerName;
21     double balance = 0;
22
23     BankAccount(this.ownerName)
24         : accountNumber = _generateAccountNumber();
25
26     void deposit(double amount) {
```

C: > Users > SoW > Desktop > FlutterSDK > flutter_2_4.dart > BankAccount > deposit

```
18 class BankAccount {
19
20
26 void deposit(double amount) {
27     if (amount <= 0) {
28         print("Invalid deposit amount");
29         return;
30     }
31     balance += amount;
32 }
33
34 void withdraw(double amount) {
35     if (amount <= 0) {
36         print("Invalid withdraw amount");
37         return;
38     }
39     if (amount > balance) {
40         print("Not enough balance");
41         return;
42     }
43     balance -= amount;
44 }
45
46 void displayBalance() {
47     print("$ownerName ($accountNumber): $balance");
48 }
49
50 static String _generateAccountNumber() {
51     final random = Random();
52     return List.generate(10, (_) => random.nextInt(10)).join();
53 }
54 }
55
56 class Bank {
57     final List<BankAccount> accounts = [];
58 }
```

C: > Users > SoW > Desktop > FlutterSDK > flutter_2_4.dart > BankAccount > deposit

```
55
56 class Bank {
57   final List<BankAccount> accounts = [];
58
59   BankAccount addAccount(String ownerName) {
60     final account = BankAccount(ownerName);
61     accounts.add(account);
62     return account;
63   }
64
65   BankAccount? findAccount(String accountNumber) {
66     for (var acc in accounts) {
67       if (acc.accountNumber == accountNumber) {
68         return acc;
69       }
70     }
71     return null;
72   }
73
74   void transfer(String fromAccountNumber, String toAccountNumber, double amount) {
75     final from = findAccount(fromAccountNumber);
76     final to = findAccount(toAccountNumber);
77
78     if (from == null || to == null) {
79       print("Account not found");
80       return;
81     }
82
83     if (amount <= 0) {
84       print("Invalid transfer amount");
85       return;
86     }
87
88     if (from.balance < amount) {
89       print("Not enough balance");
```

C: > Users > SoW > Desktop > FlutterSDK > flutter_2_4.dart > Bank > transfer

```
56 class Bank {  
74   void transfer(String fromAccountNumber, String toAccountNumber, double amount) {  
86     }  
87  
88     if (from.balance < amount) {  
89       print("Not enough balance");  
90       return;  
91     }  
92  
93     from.withdraw(amount);  
94     to.deposit(amount);  
95  
96     print("Transfer completed");  
97   }  
98 }
```

ПРОБЛЕМЫ

ВЫХОДНЫЕ ДАННЫЕ

КОНСОЛЬ ОТЛАДКИ

ТЕРМИНАЛ

ПОРТЫ

Bob (7819041233): 50.0

PS C:\>

ПРОБЛЕМЫ

ВЫХОДНЫЕ ДАННЫЕ

КОНСОЛЬ ОТЛАДКИ

ТЕРМИНАЛ

ПОРТЫ

PS C:\> dart run flutter_2_4.dart

Transfer completed

Alice (5785037693): 20.0

Bob (1770012415): 50.0

PS C:\>

5. Игра «РПГ-персонажи»

Условие:

Создайте систему классов для RPG-персонажей с использованием абстрактных классов и интерфейсов.

1. Абстрактный класс Character

a. Поля:

- i. name (имя персонажа)
- ii. health (здоровье, начальное = 100)
- iii. level (уровень, начальное = 1)

b. Абстрактные методы:

- i. attack() – возвращает урон.
- ii. defend(damage) – уменьшает здоровье на damage.

c. Общие методы:

- i. isAlive() – проверяет, жив ли персонаж (health > 0).

2. Классы персонажей:

a. Warrior:

- i. Атака: $15 + \text{level} * 2$.
- ii. Защита: уменьшает урон на 20%.

b. Mage:

- i. Атака: $10 + \text{level} * 3$.
- ii. Защита: получает полный урон, но может вылечиться на 5 HP после атаки.

c. Rogue:

- i. Атака: 20% шанс на критический удар (x3).
- ii. Защита: уворот (50% шанс избежать урона).

Дополнительно:

- d. Реализуйте бой между двумя персонажами (по очереди атакуют друг друга).

C: > Users > SoW > Desktop > FlutterSDK > flutter_2_5.dart > main

```
1  import 'dart:math';
2
   Run | Debug
3  void main() {
4      Character warrior = Warrior("Воин", 1);
5      Character mage = Mage("Маг", 1);
6
7      fight(warrior, mage);
8  }
9
10  /// ===== БОЙ =====
11  void fight(Character a, Character b) {
12      int turn = 0;
13
14      while (a.isAlive() && b.isAlive()) {
15          if (turn % 2 == 0) {
16              int damage = a.attack();
17              b.defend(damage);
18              print("${a.name} attacks ${b.name} for $damage");
19          } else {
20              int damage = b.attack();
21              a.defend(damage);
22              print("${b.name} attacks ${a.name} for $damage");
23          }
24
25          print("${a.name}: ${a.health} HP | ${b.name}: ${b.health} HP");
26          print("----");
27
28          turn++;
29      }
30
31      if (a.isAlive()) {
32          print("${a.name} wins");
33      } else {
34          print("${b.name} wins");
35      }
36  }
```

C: > Users > SoW > Desktop > FlutterSDK > flutter_2_5.dart > main

```
11 void fight(Character a, Character b) {
30
31     if (a.isAlive()) {
32         print("${a.name} wins");
33     } else {
34         print("${b.name} wins");
35     }
36 }
37
38 /// ===== CHARACTER =====
39 abstract class Character {
40     String name;
41     int health;
42     int level;
43
44     Character(this.name, {this.level = 1}) : health = 100;
45
46     int attack();
47
48     void defend(int damage);
49
50     bool isAlive() {
51         return health > 0;
52     }
53 }
54
55 /// ===== WARRIOR =====
56 class Warrior extends Character {
57     Warrior(String name, int level) : super(name, level: level);
58
59     @override
60     int attack() {
61         return 15 + level * 2;
62     }
63 }
```

C: > Users > SoW > Desktop > FlutterSDK > flutter_2_5.dart > main

```
56  class Warrior extends Character {
57
58      @override
59      int attack() {
60          return 15 + level * 2;
61      }
62
63
64      @override
65      void defend(int damage) {
66          int reduced = (damage * 0.8).round();
67          health -= reduced;
68      }
69  }
70
71  /// ===== MAGE =====
72  class Mage extends Character {
73      Mage(String name, int level) : super(name, level: level);
74
75      @override
76      int attack() {
77          return 10 + level * 3;
78      }
79
80      @override
81      void defend(int damage) {
82          health -= damage;
83          health += 5;
84      }
85  }
86
87  /// ===== ROGUE =====
88  class Rogue extends Character {
89      Rogue(String name, int level) : super(name, level: level);
90
91      @override
```


C: > Users > SoW > Desktop > FlutterSDK > flutter_2_5.dart > main

```
88   class Rogue extends Character {  
89       Rogue(String name, int level) : super(name, level, level),  
90  
91       @override  
92       int attack() {  
93           final random = Random();  
94           bool crit = random.nextDouble() < 0.2;  
95  
96           int base = 20;  
97           return crit ? base * 3 : base;  
98       }  
99  
100      @override  
101      void defend(int damage) {  
102          final random = Random();  
103          bool dodge = random.nextDouble() < 0.5;  
104  
105          if (!dodge) {  
106              health -= damage;  
107          }  
108      }  
109  }
```

```
PS C:\> dart run flutter_2_5.dart
```

```
Воин attacks Маг for 17
```

```
Воин: 100 HP | Маг: 88 HP
```

```
----
```

```
Маг attacks Воин for 13
```

```
Воин: 90 HP | Маг: 88 HP
```

```
----
```

```
Воин attacks Маг for 17
```

```
Воин: 90 HP | Маг: 76 HP
```

```
----
```

```
Маг attacks Воин for 13
```

```
Воин: 80 HP | Маг: 76 HP
```

```
----
```

```
Воин attacks Маг for 17
```

```
Воин: 80 HP | Маг: 64 HP
```

```
----
```

```
Маг attacks Воин for 13
```

```
Воин: 70 HP | Маг: 64 HP
```

```
----
```

```
Воин attacks Маг for 17
```

```
Воин: 70 HP | Маг: 52 HP
```

```
----
```

```
Маг attacks Воин for 13
```

```
Воин: 60 HP | Маг: 52 HP
```

```
----
```

```
Воин attacks Маг for 17
```

```
Воин: 60 HP | Маг: 40 HP
```

```
----
```

```
Маг attacks Воин for 13
```

```
Воин: 50 HP | Маг: 40 HP
```

```
----
```

```
Воин attacks Маг for 17
```

```
Воин: 50 HP | Маг: 28 HP
```

```
----
```

```
Маг attacks Воин for 13
```

```
Воин: 40 HP | Маг: 28 HP
```

```
----
```

Воин: 60 HP | Маг: 40 HP

Маг attacks Воин for 13

Воин: 50 HP | Маг: 40 HP

Воин attacks Маг for 17

Воин: 50 HP | Маг: 28 HP

Маг attacks Воин for 13

Воин: 40 HP | Маг: 28 HP

Воин attacks Маг for 17

Воин: 40 HP | Маг: 16 HP

Маг attacks Воин for 13

Воин: 30 HP | Маг: 16 HP

Воин attacks Маг for 17

Воин: 30 HP | Маг: 4 HP

Маг attacks Воин for 13

Воин: 20 HP | Маг: 4 HP

Воин attacks Маг for 17

Воин: 20 HP | Маг: -8 HP

Воин wins

PS C:\> █